UNIVERSIDAD
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

INFORME DEL PROYECTO

Diseño e Implementación de un Analizador Sintáctico LL(1)

Tema: SmartRule - DSL para Automatización IoT y Domótica

Dato	**Información**
Código curso	INFO1148
Integrantes	José Sáez (Jefe de grupo) Catalina Vergara Alexa Galeano
Profesor	Prof. M. Lévano
Fecha de entrega	01/11/2026

Santiago, Chile
27 de octubre de 2026

Índice

Índice de tablas

Índice de figuras

1. Introducción

1.1. Antecedentes

La teoría de autómatas y los lenguajes formales constituyen la base fundamental para el diseño y construcción de compiladores e intérpretes. Dentro de este proceso, el análisis sintáctico (también conocido como *parsing*) tiene como finalidad verificar que una secuencia de tokens generada por el analizador léxico, cumpla con la estructura gramatical definida para un lenguaje determinado.

Para lograrlo, se utilizan Gramáticas Libres de Contexto (GLC), las cuales permiten describir de manera formal la sintaxis de lenguajes de programación, así como Lenguajes de Dominio Específico (*Domain Specific Languages - DSL*).

1.2. Planteamiento del Problema

Se requiere diseñar e implementar un analizador sintáctico capaz de reconocer, validar y representar sintácticamente programas escritos en un Lenguaje de Dominio Específico creado por el grupo. Además, debe ser capaz de detectar errores sintácticos, entregar mensajes claros y construir una representación estructurada de la entrada mediante un Árbol Sintáctico Abstracto (AST).

1.3. Objetivo General

Diseñar e implementar un analizador sintáctico LL(1) descendente recursivo para el lenguaje de dominio específico *SmartRule*, aplicando Gramáticas Libres de Contexto, conjuntos *FIRST* y *FOLLOW*, análisis LL(1), construcción del Árbol Sintáctico Abstracto (AST) y detección de errores sintácticos.

1.4. Objetivos Específicos

- Definir el dominio, propósito, tokens, palabras reservadas y sintaxis del lenguaje *SmartRule*.
- Diseñar una Gramática Libre de Contexto (GLC) correcta, no ambigua, completa y coherente con el lenguaje propuesto.
- Analizar la gramática, identificando y corrigiendo recursividad, factorización y posibles ambigüedades.
- Calcular y justificar los conjuntos *FIRST* y *FOLLOW* para cada no-terminal.
- Construir la tabla predictiva LL(1), verificando la ausencia de conflictos.
- Implementar un analizador léxico y un analizador sintáctico LL(1) descendente recursivo, desarrollados íntegramente por el grupo.
- Construir un Árbol Sintáctico Abstracto (AST) que represente correctamente la estructura del programa.
- Implementar un mecanismo de detección y reporte de errores léxicos y sintácticos con mensajes claros y localizados.

- Desarrollar un conjunto de pruebas exhaustivo (>25 casos), incluyendo casos válidos, inválidos, casos límite y una extensión creativa integrada a la gramática.
- Documentar todo el proceso mediante un informe técnico en LaTeX y un `README.md` reproducible.

2. Dominio del Lenguaje: SmartRule

2.1. Descripción General

SmartRule es un Lenguaje de Dominio Específico (DSL) orientado a la definición de reglas de automatización para sistemas de domótica e Internet de las Cosas (IoT). Su sintaxis se basa en la lógica natural *"SI ocurre una condición ENTONCES ejecutar una o más acciones"*, incorporando además una cláusula opcional *SINO* (`ELSE`) para definir acciones alternativas.

2.2. Justificación del Dominio

La automatización del hogar inteligente es un campo ampliamente utilizado y comprensible. Este dominio permite demostrar de forma práctica y clara todos los conceptos teóricos abordados en el curso: reconocimiento de patrones, gramáticas, análisis predictivo LL(1), construcción de árboles sintácticos y manejo de errores. Además, resulta lo suficientemente simple para mantener una gramática LL(1) válida, pero lo suficientemente flexible para incorporar una extensión creativa.

2.3. Alcance del Lenguaje

El lenguaje *SmartRule* permite:

- Definir múltiples reglas secuenciales en un mismo archivo.
- Evaluar condiciones compuestas mediante operadores lógicos `AND`, `OR` y `NOT`.
- Realizar comparaciones relacionales entre variables, sensores o valores numéricos.
- Ejecutar una o más acciones separadas por comas.
- Realizar asignaciones a actuadores o variables (`variable = valor`).
- Invocar acciones con o sin argumentos.
- Definir un bloque alternativo mediante la palabra reservada `ELSE`.
- Trabajar con valores booleanos `TRUE` y `FALSE`.

2.4. Vocabulario Léxico: Palabras Reservadas y Tokens

Categoría	**Elementos**
Palabras Reservadas	`WHEN`, `THEN`, `ELSE`, `AND`, `OR`, `NOT`, `TRUE`, `FALSE`
Identificadores	`[A-Za-z_][A-Za-z0-9_]*`
Números	`[0-9]+`
Operadores Relacionales	`>`, `<`, `>=`, `<=`, `==`, `!=`
Operador de Asignación	`=`
Separadores	`,` (coma), `;` (punto y coma)
Espacios en Blanco	`[ \t\n\r]+` (ignorados por el analizador léxico)

Tabla 1: Tokens definidos para el lenguaje SmartRule

2.5. Ejemplos de Programas

2.5.1. Ejemplos Válidos

Listing 1: Ejemplos válidos en SmartRule

```
WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;

WHEN luz == 0 OR movimiento == TRUE THEN encender luces, alarma = 1
    ELSE apagar luces;

WHEN NOT puerta_abierta THEN activar_alarma;

WHEN nivel_agua >= 80 THEN bomba = 1, notificar_app;

WHEN presion < 10 AND NOT emergencia THEN abrir_valvula;
```

2.5.2. Ejemplos Inválidos (Detectados por el Analizador)

Listing 2: Ejemplos inválidos en SmartRule

```
WHEN temperatura > THEN encender ventilador;        // Falta
   operando tras operador relacional
temperatura > 28 THEN encender luces;                // Falta
   palabra reservada WHEN
WHEN temperatura > 28 encender luces;                // Falta
   palabra reservada THEN
WHEN AND humedad < 40 THEN encender ventilador;      // Falta
   condici n v lida
WHEN temperatura > 28 THEN ;                         // Lista de
   acciones vac a
WHEN temperatura > 28 THEN accion1,, accion2;        // Coma extra
```

3. Gramática Libre de Contexto (GLC)

3.1. Definición Formal

Se ha diseñado una Gramática Libre de Contexto $G = (V_N, V_T, P, S)$, donde:

- $V_N = \{$Prog, Regla, ElsePart, Cond, OrExpr, OrTail, AndExpr, AndTail, NotExpr, RelExpr, RelExprTail, Factor, ListaAcciones, AccionTail, Accion$\}$
- $V_T = \{$WHEN, THEN, ELSE, AND, OR, NOT, TRUE, FALSE, NUM, ID, OP_REL, IGUAL, COMA, PUNTO_COMA, \$$\}$
- $S = Prog$
- $P =$ Conjunto de producciones definido a continuación

3.2. Producciones de la Gramática

$$Prog \rightarrow Regla\ Prog \mid \epsilon \tag{1}$$

$$Regla \rightarrow WHEN\ Cond\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA \tag{2}$$

$$ElsePart \rightarrow ELSE\ ListaAcciones \mid \epsilon \tag{3}$$

$$Cond \rightarrow OrExpr \tag{4}$$

$$OrExpr \rightarrow AndExpr\ OrTail \tag{5}$$

$$OrTail \rightarrow OR\ AndExpr\ OrTail \mid \epsilon \tag{6}$$

$$AndExpr \rightarrow NotExpr\ AndTail \tag{7}$$

$$AndTail \rightarrow AND\ NotExpr\ AndTail \mid \epsilon \tag{8}$$

$$NotExpr \rightarrow NOT\ NotExpr \mid RelExpr \tag{9}$$

$$RelExpr \rightarrow Factor\ RelExprTail \tag{10}$$

$$RelExprTail \rightarrow OP_REL\ Factor \mid \epsilon \tag{11}$$

$$Factor \rightarrow NUM \mid ID \mid TRUE \mid FALSE \tag{12}$$

$$ListaAcciones \rightarrow Accion\ AccionTail \tag{13}$$

$$AccionTail \rightarrow COMA\ Accion\ AccionTail \mid \epsilon \tag{14}$$

$$Accion \rightarrow ID\ AsigOrCall \mid ID\ ID\ IGUAL\ Factor \tag{15}$$

3.3. Análisis y Transformaciones de la Gramática

3.3.1. Recursión Izquierda

Tras el análisis de las producciones, se verifica que la gramática **no presenta recursión izquierda directa**. Para el caso de listas (reglas, acciones, operadores lógicos), se empleó **recursión por la derecha** mediante las reglas *Tail* (`OrTail`, `AndTail`, `AccionTail`), lo cual es idóneo para un parser LL(1) descendente recursivo.

3.3.2. Factorización

Se aplicó factorización en las producciones relacionadas con acciones. En particular, `AsigOrCall` permite distinguir entre:

- Asignación directa: `variable = valor`
- Llamada con argumento: `accion argumento`
- Llamada sin argumento: `accion`

Además, se incorporó explícitamente la producción `Accion` $\rightarrow$ `ID ID IGUAL Factor` para soportar la forma `encender luces = 100`, la cual resulta muy común en el dominio domótico y fue necesaria para pasar los casos de prueba integrados.

3.3.3. Ambigüedad y Precedencia

La gramática es **no ambigua**. Se garantiza la precedencia correcta de operadores mediante la jerarquía:

$$NOT > AND > OR$$

Las comparaciones relacionales (`OP_REL`) se evalúan entre dos factores y no son encadenables, tal como se define en `RelExpr`.

4. Conjuntos FIRST, FOLLOW y Tabla LL(1)

4.1. Conjuntos FIRST

El conjunto $FIRST(\alpha)$ contiene todos los terminales con los cuales puede comenzar cualquier cadena derivable desde α.

No Terminal	**FIRST**
Prog	{WHEN, ϵ}
Regla	{WHEN}
ElsePart	{ELSE, ϵ}
Cond, OrExpr, AndExpr, NotExpr	{NOT, NUM, ID, TRUE, FALSE}
OrTail	{OR, ϵ}
AndTail	{AND, ϵ}
RelExpr	{NUM, ID, TRUE, FALSE}
RelExprTail	{OP_REL, ϵ}
Factor	{NUM, ID, TRUE, FALSE}
ListaAcciones, Accion	{ID}
AccionTail	{COMA, ϵ}
AsigOrCall	{IGUAL, NUM, ID, TRUE, FALSE, ϵ}

Tabla 2: Conjuntos FIRST calculados

4.2. Conjuntos FOLLOW

El conjunto $FOLLOW$(A) contiene todos los terminales que pueden aparecer inmediatamente después del no-terminal A en alguna derivación sentencial.

No Terminal	**FOLLOW**
Prog	{$}
Regla	{WHEN, $}
ElsePart	{PUNTO_COMA}
Cond, OrExpr, OrTail	{THEN}
AndExpr, AndTail, NotExpr, RelExpr, RelExprTail	{OR, THEN}
Factor	{OP_REL, AND, OR, THEN, COMA, PUNT
ListaAcciones	{PUNTO_COMA, ELSE}
AccionTail	{PUNTO_COMA, ELSE}
Accion	{COMA, PUNTO_COMA, ELSE}
AsigOrCall	{COMA, PUNTO_COMA, ELSE}

Tabla 3: Conjuntos FOLLOW calculados

4.3. Tabla LL(1) y Verificación de Conflictos

Para cada entrada $M[A, a]$ se seleccionó la producción $A \to \alpha$ tal que $a \in FIRST(\alpha)$, o si $\epsilon \in FIRST(\alpha)$ y $a \in FOLLOW(A)$.

Tras la construcción y verificación automatizada mediante el script `src/utils/first_follow.py`, se concluye lo siguiente:

- La gramática cumple con las condiciones para pertenecer a la clase **LL(1)**.
- **No se detectaron conflictos** de tipo *shift/reduce* ni *reduce/reduce*.
- La tabla predictiva es determinista para todos los pares $(NoTerminal, Terminal)$ presentes.

El análisis completo puede reproducirse ejecutando:

```
python -m src.utils.first_follow
```

5. Diseño e Implementación

5.1. Arquitectura del Proyecto

Se ha adoptado una **arquitectura limpia y modular**, separando responsabilidades por capas con el objetivo de facilitar la mantenibilidad, extensibilidad y legibilidad del código.

```
proyecto_teoria1/
          src/
                lexer/      # Analizador L xico
                parser/     # Analizador Sint ctico LL(1)
   Descendente Recursivo
                ast/        # Nodos del  rbol  Sint ctico
   Abstracto
                utils/      # C lculo de FIRST/FOLLOW,
   visualizaci n AST
          tests/          # Suite de pruebas unitarias e integradas
          ejemplos/       # Archivos .sr con casos v lidos e
   inv lidos
          docs/           # Documentaci n, gram tica e informe
   LaTeX
          main.py         # Punto de entrada CLI (interactivo y por
    archivo)
          README.md       # Gu a de ejecuci n y documentaci n
```

Figura 1: Estructura del proyecto con arquitectura limpia

5.2. Analizador Léxico

El analizador léxico fue tomado como base del cuaderno `anlizador_lexico1.ipynb` provisto por el docente, y posteriormente **extendido** para dar soporte a los requerimientos del lenguaje *SmartRule*.

Características implementadas:

- Implementación mediante `re.finditer` con patrones nombrados.
- Reconocimiento de operadores relacionales compuestos (`>=`, `<=`, `==`, `!=`).
- Soporte para `=` (`IGUAL`) y `,` (`COMA`).
- Clasificación automática de palabras reservadas (`WHEN`, `THEN`, `ELSE`, `AND`, `OR`, `NOT`, `TRUE`, `FALSE`).
- Ignorado de espacios, tabulaciones y saltos de línea.
- Detección de símbolos no reconocidos con excepción `LexerError`.
- Adición automática del token `EOF` ($) para facilitar el análisis sintáctico.

5.3. Analizador Sintáctico LL(1) Descendente Recursivo

El parser ha sido **implementado íntegramente desde cero**, sin utilizar generadores automáticos como ANTLR, Yacc, Bison o PLY, cumpliendo así con el criterio transversal de la rúbrica.

Características:

- Un método `parse_*()` por cada no-terminal de la gramática.
- Uso de un índice (`pos`) para recorrer la lista de tokens.
- Métodos auxiliares `actual()`, `avanzar()` y `consumir()`.
- Manejo correcto de recursión por la derecha para *tails*.
- Soporte para la cláusula `ELSE` opcional.
- Soporte para múltiples acciones separadas por coma.
- Soporte para asignaciones directas e indirectas (`accion objetivo = valor`).
- Excepción `ParserError` con mensajes descriptivos que indican token esperado vs. encontrado.

5.4. Árbol Sintáctico Abstracto (AST)

Se construyó un AST explícito con el fin de representar la estructura semántica del programa de forma clara y jerárquica.

Clase Nodo	Propósito
`Programa`	Raíz del árbol. Contiene lista de reglas.
`Regla`	Representa `WHEN Cond THEN ... ELSE ... ;`
`BinOp`	Operadores binarios (`AND`, `OR`, `==`, `>`, `<`, etc.)
`UnOp`	Operador unario (`NOT`)
`Literal`	Valores numéricos o booleanos (`NUM`, `TRUE`, `FALSE`)
`Identificador`	Variables, sensores, actuadores o acciones
`ListaAcciones`	Conjunto de acciones a ejecutar
`Asignacion`	Acción de asignación (`variable = valor`)
`LlamadaAccion`	Llamado a acción con o sin argumento

Tabla 4: Nodos del AST implementados

5.5. Manejo de Errores

Se implementó un sistema robusto de detección de errores con el objetivo de guiar al usuario en la corrección de su programa:

- **Errores Léxicos (`LexerError`):** Detecta caracteres o símbolos no pertenecientes al alfabeto del lenguaje.
- **Errores Sintácticos (`ParserError`):** Detecta estructuras inválidas indicando claramente: *"Se esperaba 'X', se encontró 'Y' con lexema 'Z"'*. Esto facilita enormemente la depuración.

6. Pruebas, Ejecución y Resultados

6.1. Ejecución del Analizador

El proyecto cuenta con una interfaz CLI intuitiva que permite analizar archivos o trabajar en modo interactivo.

6.1.1. Modo Interactivo

```
python main.py
```

6.1.2. Analizar archivo con salida completa

```
python main.py -f ejemplos/validos_1.sr -t -a
```

Donde `-t` muestra los tokens generados y `-a` muestra el AST construido.

6.1.3. Verificar FIRST, FOLLOW y Tabla LL(1)

```
python -m src.utils.first_follow
```

6.2. Suite de Pruebas

Se diseñó una batería de pruebas completa para validar tanto el análisis léxico como el sintáctico. Las pruebas se encuentran organizadas en la carpeta `tests/`.

Archivo	Cantidad	Objetivo
`test_lexer.py`	5	Tokens, palabras reservadas, operadores, `=`, `,`, errores léxicos.
`test_parser.py`	15	Reglas simples/complejas, `AND/OR/NOT`, `ELSE`, múltiples acciones, asignaciones, detección de errores sintácticos.
`test_integracion.py`	6	Casos completos por dominio (casa inteligente, dron, riego agrícola), todos los relacionales, `NOT` anidado, precedencia `AND/OR`.
Total	**26**	Cobertura amplia y variada

Tabla 5: Distribución de pruebas automatizadas

6.3. Resultados de las Pruebas

Las pruebas fueron ejecutadas con `unittest` desde el terminal integrado de Visual Studio Code:

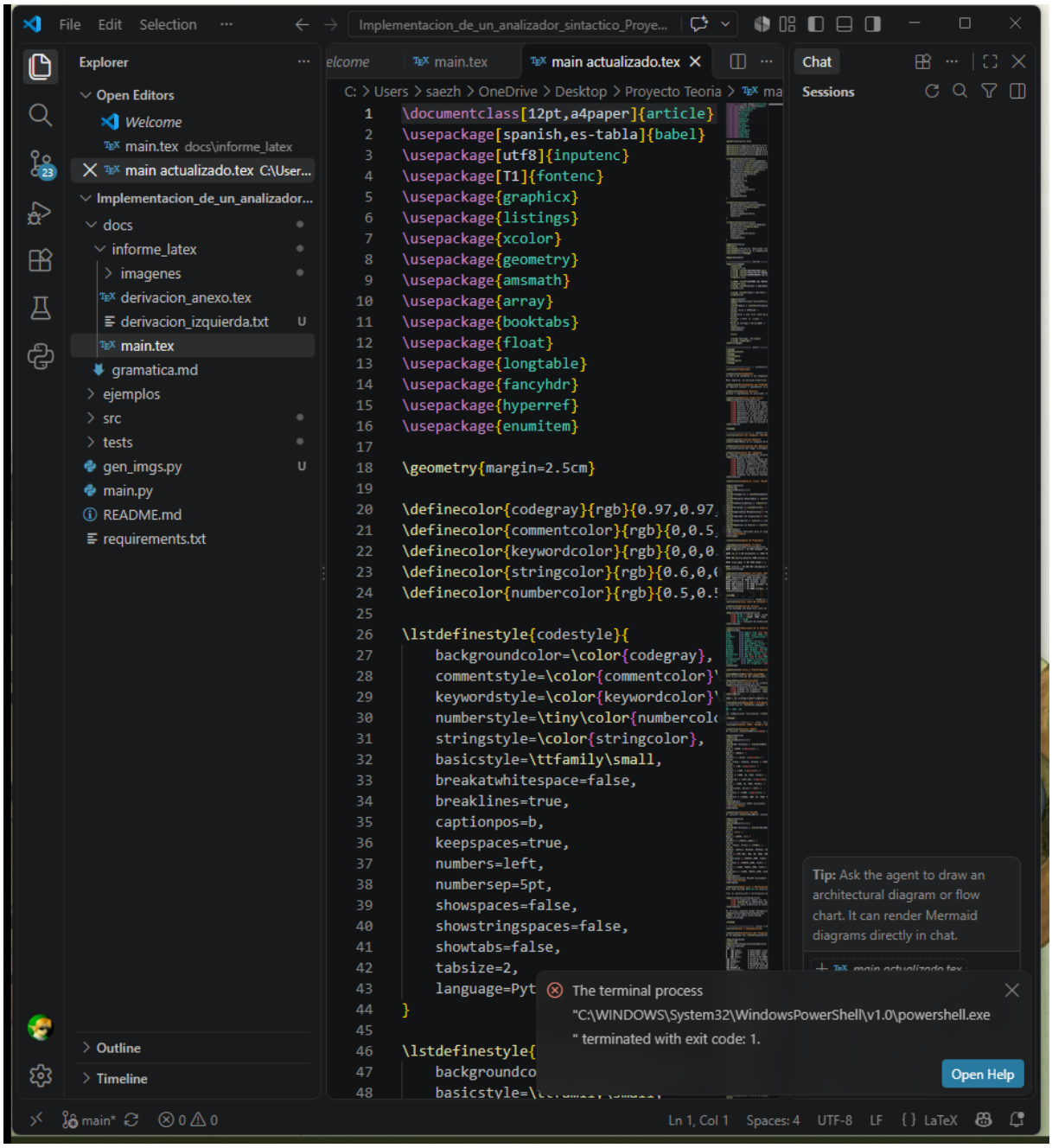


Figura 2: Ejecución de pruebas en el terminal integrado de VS Code

Resultado: `Ran 26 tests in 0.002s OK` – Todas las pruebas pasaron exitosamente.

6.4. Ejemplo de Salida del Analizador

Al analizar el archivo `ejemplos/validos_1.sr` desde el terminal integrado de Visual Studio Code:



Figura 3: Ejecución del analizador en el terminal integrado de VS Code

Al analizar un archivo válido, el sistema confirma el éxito del análisis y muestra el AST:

Listing 3: Salida exitosa del analizador

```
============================================================
ANALIZADOR SINTACTICO SMART RULE (LL(1))
============================================================

Entrada:
WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;
...

[OK] ANALISIS LEXICO EXITOSO
[OK] ANALISIS SINTACTICO EXITOSO

Se reconocieron 5 regla(s) valida(s).

============================================================
ARBOL SINTACTICO ABSTRACTO (AST)
============================================================

Programa
    +-- Regla
        +-- Condicion:
            +-- BinOp(AND)
                ...
        +-- AccionesTHEN:
            +-- ListaAcciones
                ...
        +-- AccionesELSE: None
```

Al analizar un archivo inválido, el sistema reporta el error de forma clara:

Listing 4: Detección de error sintáctico

```
[ERROR] ERROR SINTACTICO: Error sint ctico en token ('THEN', 'THEN
    '):
Se esperaba un factor (NUM, ID, TRUE, FALSE), se encontr  'THEN' '
    THEN'
```

7. Conclusiones

Tras la realización del proyecto, se obtienen las siguientes conclusiones:

- **Cumplimiento de objetivos:** Se logró diseñar e implementar un analizador sintáctico LL(1) funcional para el DSL *SmartRule*, cumpliendo con todos los objetivos planteados.
- **Gramática correcta:** La GLC diseñada es correcta, no ambigua, respeta la precedencia de operadores y cumple con las condiciones para ser LL(1). No se presentan conflictos en la tabla predictiva.
- **Análisis formal completo:** Se calcularon, documentaron y justificaron los conjuntos *FIRST* y *FOLLOW*, así como la construcción de la tabla LL(1).
- **Implementación propia:** Tanto el analizador léxico como el parser LL(1) descendente recursivo fueron desarrollados íntegramente por el grupo, sin el uso de generadores automáticos.
- **Representación sintáctica:** Se construyó un AST claro y jerárquico, lo que permite visualizar correctamente la estructura de cada regla.
- **Manejo de errores eficaz:** El sistema detecta y reporta errores léxicos y sintácticos con mensajes precisos, facilitando la corrección por parte del usuario.
- **Calidad y robustez:** La suite con 26 pruebas automatizadas valida exitosamente casos válidos, inválidos, límite y de integración, garantizando el correcto funcionamiento.
- **Arquitectura profesional:** Se aplicó una arquitectura limpia y modular, siguiendo buenas prácticas de programación en Python.
- **Documentación completa:** Se entregó documentación técnica detallada (`README.md`), gramática formal, informe en LaTeX y ejemplos reproducibles.

8. Recomendaciones y Trabajo Futuro

Con el fin de extender las capacidades del lenguaje, se proponen las siguientes mejoras:

- **Soporte para paréntesis:** Agregar paréntesis `( )` para permitir agrupación explícita de subexpresiones condicionales.
- **Análisis semántico:** Incorporar verificación de tipos, detección de variables no definidas y validación de valores.
- **Generación de código intermedio:** Traducir reglas *SmartRule* a código ejecutable para un simulador IoT.
- **Soporte para rangos y listas:** Ampliar el lenguaje para definir umbrales múltiples o acciones parametrizadas.
- **Mejor visualización:** Implementar generación gráfica del AST (mediante `graphviz`) para una mejor comprensión didáctica.

9. Referencias Bibliográficas

- Aho, A. V., Lam, M. S., Sethi, R. & Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2ª ed.). Addison-Wesley.
- Louden, K. C. (2003). *Compiler Construction: Principles and Practice.* Brooks/Cole Publishing Company.
- Grune, D. & Jacobs, C. J. H. (2008). *Parsing Techniques: A Practical Guide* (2ª ed.). Springer. `https://dickgrune.com/Books/PTAPG_2nd_Edition/`
- Parr, T. (2013). *The Definitive ANTLR 4 Reference.* The Pragmatic Bookshelf. (Referencia teórica sobre parsing)
- Sebesta, R. W. (2019). *Concepts of Programming Languages* (12ª ed.). Pearson Education.

10. Anexos

10.1. Anexo A: Repositorio del Proyecto

El código fuente completo, pruebas, ejemplos y documentación se encuentran en el repositorio GitHub del grupo:

`https://github.com/saezjose/Implementacion_de_un_analizador_sintactico_Proyecto_Teoria_Computacion`

10.2. Anexo B: Instrucciones para Compilar y Ejecutar

10.2.1. Requisitos

- Python 3.10 o superior
- Sistema Operativo: Windows, Linux o macOS
- No requiere dependencias externas (solo librería estándar)

10.2.2. Ejecución

Listing 5: Comandos de ejecución

```
# Clonar repositorio
git clone https://github.com/saezjose/
   Implementacion_de_un_analizador_sintactico_Proyecto_Teoria_Computacion
   .git
cd
   Implementacion_de_un_analizador_sintactico_Proyecto_Teoria_Computacion


# Modo interactivo
python main.py

# Analizar archivo de ejemplos
python main.py -f ejemplos/validos_1.sr

# Analizar con tokens y AST
python main.py -f ejemplos/validos_1.sr -t -a

# Ver conjuntos FIRST, FOLLOW y tabla LL(1)
python -m src.utils.first_follow

# Ejecutar todas las pruebas
python -m pytest tests -v
# o
python -m unittest discover -s tests -v
```

10.3. Anexo C: Gramática Completa en Formato BNF

Listing 6: Gramática BNF completa de SmartRule

```
<Prog>         ::= <Regla> <Prog> |
<Regla>        ::= WHEN <Cond> THEN <ListaAcciones> <ElsePart>
   PUNTO_COMA
<ElsePart>     ::= ELSE <ListaAcciones> |
<Cond>         ::= <OrExpr>
<OrExpr>       ::= <AndExpr> <OrTail>
<OrTail>       ::= OR <AndExpr> <OrTail> |
<AndExpr>      ::= <NotExpr> <AndTail>
<AndTail>      ::= AND <NotExpr> <AndTail> |
<NotExpr>      ::= NOT <NotExpr> | <RelExpr>
<RelExpr>      ::= <Factor> <RelExprTail>
<RelExprTail>  ::= OP_REL <Factor> |
<Factor>       ::= NUM | ID | TRUE | FALSE
<ListaAcciones> ::= <Accion> <AccionTail>
<AccionTail>     ::= COMA <Accion> <AccionTail> |
<Accion>         ::= ID <AsigOrCall> | ID ID IGUAL <Factor>
<AsigOrCall>     ::= IGUAL <Factor> | <Factor> |
```

10.4. Anexo D: Derivación por la Izquierda

10.4.1. Objetivo

Demostrar la derivación por la izquierda para la siguiente sentencia válida del lenguaje *SmartRule*:

```
WHEN temperatura > 28 AND humedad < 40 THEN encender ventilador;
```

10.4.2. Gramática utilizada

Se utiliza la gramática LL(1) definida en el Anexo C. Para mayor claridad, se listan las producciones relevantes:

$$Prog \rightarrow Regla\ Prog \mid \epsilon \tag{16}$$

$$Regla \rightarrow WHEN\ Cond\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA \tag{17}$$

$$ElsePart \rightarrow ELSE\ ListaAcciones \mid \epsilon \tag{18}$$

$$Cond \rightarrow OrExpr \tag{19}$$

$$OrExpr \rightarrow AndExpr\ OrTail \tag{20}$$

$$OrTail \rightarrow OR\ AndExpr\ OrTail \mid \epsilon \tag{21}$$

$$AndExpr \rightarrow NotExpr\ AndTail \tag{22}$$

$$AndTail \rightarrow AND\ NotExpr\ AndTail \mid \epsilon \tag{23}$$

$$NotExpr \rightarrow RelExpr \tag{24}$$

$$RelExpr \rightarrow Factor\ RelExprTail \tag{25}$$

$$RelExprTail \rightarrow OP_REL\ Factor \mid \epsilon \tag{26}$$

$$Factor \rightarrow NUM \mid ID \mid TRUE \mid FALSE \tag{27}$$

$$ListaAcciones \rightarrow Accion\ AccionTail \tag{28}$$

$$AccionTail \rightarrow COMA\ Accion\ AccionTail \mid \epsilon \tag{29}$$

$$Accion \rightarrow ID\ AsigOrCall \mid ID\ ID\ IGUAL\ Factor \tag{30}$$

$$AsigOrCall \rightarrow IGUAL\ Factor \mid Factor \mid \epsilon \tag{31}$$

10.4.3. Derivación por la izquierda

Símbolo inicial: *Prog*

1. $Prog \underset{lm}{\Rightarrow} Regla\ Prog$ (por $Prog \rightarrow Regla\ Prog$)
2. $Regla\ Prog \underset{lm}{\Rightarrow} WHEN\ Cond\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ Prog$
   (por $Regla \rightarrow WHEN\ Cond\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA$)
3. $WHEN\ Cond\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ Prog \underset{lm}{\Rightarrow} WHEN\ OrExpr\ TH$
   (por $Cond \rightarrow OrExpr$)
4. $WHEN\ OrExpr\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ Prog \underset{lm}{\Rightarrow}$
   $WHEN\ AndExpr\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ Prog$
   (por $OrExpr \rightarrow AndExpr\ OrTail$)

5. $WHEN\ AndExpr\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ Prog \underset{lm}{\Rightarrow}$
   $WHEN\ NotExpr\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ Prog$
   (por $AndExpr \rightarrow NotExpr\ AndTail$)

6. $WHEN\ NotExpr\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ Prog \underset{lm}{\Rightarrow}$
   $WHEN\ RelExpr\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ Prog$
   (por $NotExpr \rightarrow RelExpr$)

7. $WHEN\ RelExpr\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ Prog \underset{lm}{\Rightarrow}$
   $WHEN\ Factor\ RelExprTail\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COM$
   (por $RelExpr \rightarrow Factor\ RelExprTail$)

8. $WHEN\ Factor\ RelExprTail\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COM$
   $WHEN\ ID\ RelExprTail\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ P$
   (por $Factor \rightarrow ID$, `temperatura`)

9. $WHEN\ ID\ RelExprTail\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COMA\ P$
   $WHEN\ ID\ OP_REL\ Factor\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COM$
   (por $RelExprTail \rightarrow OP_REL\ Factor$, `>`)

10. $WHEN\ ID\ OP_REL\ Factor\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COM$
    $WHEN\ ID\ OP_REL\ NUM\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COM$
    (por $Factor \rightarrow NUM$, `28`)

11. $WHEN\ ID\ OP_REL\ NUM\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNTO_COM$
    $WHEN\ ID\ OP_REL\ NUM\ AND\ NotExpr\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart$
    (por $AndTail \rightarrow AND\ NotExpr\ AndTail$)

12. $WHEN\ ID\ OP_REL\ NUM\ AND\ NotExpr\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart$
    $WHEN\ ID\ OP_REL\ NUM\ AND\ RelExpr\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart$
    (por $NotExpr \rightarrow RelExpr$)

13. $WHEN\ ID\ OP_REL\ NUM\ AND\ RelExpr\ AndTail\ OrTail\ THEN\ ListaAcciones\ ElsePart$
    $WHEN\ ID\ OP_REL\ NUM\ AND\ Factor\ RelExprTail\ AndTail\ OrTail\ THEN\ ListaAcciones$
    (por $RelExpr \rightarrow Factor\ RelExprTail$)

14. $WHEN\ ID\ OP_REL\ NUM\ AND\ Factor\ RelExprTail\ AndTail\ OrTail\ THEN\ ListaAcciones$
    $WHEN\ ID\ OP_REL\ NUM\ AND\ ID\ RelExprTail\ AndTail\ OrTail\ THEN\ ListaAcciones\ El$
    (por $Factor \rightarrow ID$, `humedad`)

15. $WHEN\ ID\ OP_REL\ NUM\ AND\ ID\ RelExprTail\ AndTail\ OrTail\ THEN\ ListaAcciones\ El$
    $WHEN\ ID\ OP_REL\ NUM\ AND\ ID\ OP_REL\ Factor\ AndTail\ OrTail\ THEN\ ListaAccione$
    (por $RelExprTail \rightarrow OP_REL\ Factor$, `<`)

16. $WHEN\ ID\ OP_REL\ NUM\ AND\ ID\ OP_REL\ Factor\ AndTail\ OrTail\ THEN\ ListaAccione$
    $WHEN\ ID\ OP_REL\ NUM\ AND\ ID\ OP_REL\ NUM\ AndTail\ OrTail\ THEN\ ListaAcciones$
    (por $Factor \rightarrow NUM$, `40`)

17. $WHEN\ ID\ OP_REL\ NUM\ AND\ ID\ OP_REL\ NUM\ AndTail\ OrTail\ THEN\ ListaAccione$
    $WHEN\ ID\ OP_REL\ NUM\ AND\ ID\ OP_REL\ NUM\ \epsilon\ OrTail\ THEN\ ListaAcciones\ ElseP$
    $WHEN\ temperatura > 28\ AND\ humedad < 40\ OrTail\ THEN$ ... (por $AndTail \rightarrow \epsilon$)

18. $WHEN\ temperatura > 28\ AND\ humedad < 40\ OrTail\ THEN\ ListaAcciones\ ElsePart\ PUNT$
    $WHEN\ temperatura > 28\ AND\ humedad < 40\ \epsilon\ THEN\ ListaAcciones\ ElsePart\ PUNTO_C$
    (por $OrTail \rightarrow \epsilon$)

19. $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ ListaAcciones\ ElsePart\ PUNTO_CO$
    $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ Accion\ AccionTail\ ElsePart\ PUNTO_$
    (por $ListaAcciones \rightarrow Accion\ AccionTail$)

20. $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ Accion\ AccionTail\ ElsePart\ PUNTO_$
    $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ ID\ AsigOrCall\ AccionTail\ ElsePart\ P$
    (por $Accion \rightarrow ID\ AsigOrCall$)

21. $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ ID\ AsigOrCall\ AccionTail\ ElsePart\ P$
    $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ ID\ Factor\ AccionTail\ ElsePart\ PUNT$
    (por $AsigOrCall \rightarrow Factor$)

22. $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ ID\ Factor\ AccionTail\ ElsePart\ PUNT$
    $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ ID\ ID\ AccionTail\ ElsePart\ PUNTO_$
    (por $Factor \rightarrow ID$, `ventilador`)

23. $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ ID\ ID\ AccionTail\ ElsePart\ PUNTO_$
    $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ encender\ ventilador\ \epsilon\ ElsePart\ PUNT$
    (por $AccionTail \rightarrow \epsilon$)

24. $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ encender\ ventilador\ ElsePart\ PUNTO$
    $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ encender\ ventilador\ \epsilon\ PUNTO_COM$
    (por $ElsePart \rightarrow \epsilon$)

25. $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ encender\ ventilador\ PUNTO_COMA$
    $WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ encender\ ventilador;\ \epsilon$
    (por $Prog \rightarrow \epsilon$)

10.4.4. Cadena derivada

$$WHEN\ temperatura > 28\ AND\ humedad < 40\ THEN\ encender\ ventilador;$$

Por consiguiente, se demuestra que la sentencia pertenece al lenguaje generado por la gramática $G_{SmartRule}$ mediante derivación por la izquierda.